
Fast Log-Domain Sinkhorn Optimal Transport with Warp-Level GPU Reductions

Hao Xiao
ATLAS AI Lab
xiaohao@atlasthinktank.com

Abstract

Entropic regularized optimal transport (OT) via the Sinkhorn algorithm has become a fundamental tool in machine learning, yet existing implementations either suffer from numerical instability for small regularization parameters or incur significant overhead from deep learning frameworks. We present FASTSINKHORN, a lightweight, native CUDA implementation of the log-domain Sinkhorn algorithm that combines warp-level shuffle reductions with shared-memory tiling to achieve high GPU utilization without sacrificing numerical stability. Our solver operates entirely in the log-domain, enabling robust computation for regularization parameters as small as $\varepsilon = 10^{-4}$ where standard-domain methods fail. On dense OT problems with $n = m = 8192$, our implementation achieves $12\times$ speedup over the widely-used POT library and $5.9\times$ speedup over GPU-accelerated PyTorch baselines, while consuming only 256 MB of GPU memory. We validate our solver on image color transfer, 3D point cloud matching, and convergence analysis, demonstrating that native CUDA kernels with careful numerical treatment provide a practical and efficient foundation for large-scale optimal transport computation.

1 Introduction

Optimal transport (OT) provides a mathematically principled framework for comparing probability distributions by computing the minimum-cost coupling between them [Villani, 2003, 2009]. In the discrete setting, the Kantorovich formulation reduces to a linear program over coupling matrices, with applications spanning generative modeling [Arjovsky et al., 2017], domain adaptation [Courty et al., 2017], document similarity [Kusner et al., 2015], and geometric processing [Solomon et al., 2015].

The computational bottleneck of exact OT solvers—scaling as $O(n^3 \log n)$ via the network simplex algorithm—was dramatically alleviated by Cuturi [2013], who introduced entropic regularization and showed that the resulting problem can be solved via Sinkhorn’s matrix scaling algorithm in near-linear time per iteration. This breakthrough has led to widespread adoption of the Sinkhorn algorithm in machine learning pipelines [Peyré and Cuturi, 2019].

Despite its popularity, practical deployment of the Sinkhorn algorithm on modern hardware faces two key challenges:

Numerical instability. The standard-domain Sinkhorn algorithm computes the Gibbs kernel $K_{ij} = \exp(-C_{ij}/\varepsilon)$, which causes catastrophic overflow or underflow in floating-point arithmetic when the regularization parameter ε is small—precisely the regime needed for accurate OT approximation. Log-domain stabilization [Schmitzer, 2019] addresses this but has not been widely adopted in GPU-optimized implementations.

Framework overhead. Existing GPU implementations rely on deep learning frameworks (PyTorch, JAX) or specialized libraries (KeOps/GeomLoss [Feydy et al., 2019, Charlier et al., 2021]), which introduce significant overhead from automatic differentiation graphs, memory allocators, and Python-to-GPU dispatch. For applications that require only the forward OT computation—such as point cloud registration, color transfer, or batch distance computation—this overhead is unnecessary.

1.1 Contributions

We present FASTSINKHORN, a lightweight CUDA C++ library for entropic regularized optimal transport. Our contributions are:

1. **Numerically stable GPU solver.** A log-domain Sinkhorn implementation with a two-pass LogSumExp reduction that operates entirely in log-space, handling regularization parameters as small as $\varepsilon = 10^{-4}$ without numerical failure.
2. **Warp-level GPU optimizations.** We exploit CUDA warp shuffle instructions (`_shfl_down_sync`) for intra-warp reductions, combined with shared memory for cross-warp communication. This hierarchical reduction eliminates shared-memory bank conflicts within warps and reduces synchronization overhead by $1.93\times$ compared to pure shared-memory reductions.
3. **Comprehensive evaluation.** We provide extensive benchmarks against POT [Flamary et al., 2021], GeomLoss [Feydy et al., 2019], and PyTorch-based solvers, along with ablation studies quantifying the contribution of each optimization, numerical stability analysis, and real-world applications in image color transfer and point cloud matching.

Our solver is self-contained (no dependencies beyond the CUDA runtime), supports both pre-computed cost matrices and Euclidean point cloud inputs, and is publicly available under the MIT license.

2 Related Work

Entropic optimal transport. The use of entropic regularization for optimal transport was popularized by Cuturi [2013], who showed that adding a KL-divergence penalty transforms the linear program into a strictly convex problem solvable via Sinkhorn iterations [Sinkhorn, 1967]. The comprehensive treatment by Peyré and Cuturi [2019] establishes the theoretical foundations, including convergence rates and approximation guarantees. Altschuler et al. [2017] provided near-linear time complexity bounds, showing that $\tilde{O}(n^2/\varepsilon^2)$ operations suffice for an ε -approximate solution.

Numerical stabilization. The standard Sinkhorn algorithm suffers from numerical overflow/underflow when ε is small, as the Gibbs kernel entries $K_{ij} = e^{-C_{ij}/\varepsilon}$ span extreme ranges. Schmitzer [2019] proposed log-domain stabilization, and Chizat et al. [2018] extended this to unbalanced settings. While these stabilizations are well-understood theoretically, many GPU implementations still default to the numerically fragile standard-domain formulation.

GPU-accelerated solvers. Several GPU-based OT solvers exist in the literature. The POT library [Flamary et al., 2021] provides a comprehensive Python API with optional GPU acceleration via CuPy. GeomLoss [Feydy et al., 2019] and its KeOps backend [Charlier et al., 2021] offer GPU-accelerated Sinkhorn divergences with automatic differentiation, designed for integration with PyTorch. These framework-based approaches provide flexibility but carry overhead from the Python runtime, autograd engine, and generic GPU memory management.

Algorithmic acceleration. Beyond GPU parallelization, several algorithmic improvements reduce the computational cost of OT. Multiscale approaches [Gerber and Maggioni, 2017, Schmitzer, 2019] use coarse-to-fine strategies with sparse representations. Low-rank factorizations [Scetbon et al., 2021] approximate the transport plan with reduced-rank factors, achieving sub-quadratic complexity. Accelerated gradient methods [Dvurechensky et al., 2018, Lin et al., 2019] provide improved convergence rates for the dual problem. These approaches are complementary to our GPU optimization strategy and could be combined in future work.

Our position. Our work occupies a distinct niche: a *native CUDA* implementation that combines log-domain stability with warp-level GPU optimizations, targeting the common case of dense, moderate-sized ($n, m \leq 16384$) OT problems where the full cost matrix fits in GPU memory. Unlike framework-based approaches, our solver has zero Python overhead and minimal memory footprint, making it suitable for latency-sensitive applications and embedding in C++ pipelines.

3 Background

3.1 Discrete Optimal Transport

Let $\mu = (\mu_1, \dots, \mu_n) \in \Sigma_n$ and $\nu = (\nu_1, \dots, \nu_m) \in \Sigma_m$ be discrete probability distributions on point sets $\{x_i\}_{i=1}^n$ and $\{y_j\}_{j=1}^m$, where $\Sigma_k = \{a \in \mathbb{R}_+^k : \sum_i a_i = 1\}$ denotes the probability simplex. Given a cost matrix $C \in \mathbb{R}_+^{n \times m}$ with $C_{ij} = c(x_i, y_j)$, the *Kantorovich optimal transport problem* seeks the minimum-cost coupling:

$$\mathcal{T}_c(\mu, \nu) = \min_{\pi \in \Pi(\mu, \nu)} \langle C, \pi \rangle_F = \min_{\pi \in \Pi(\mu, \nu)} \sum_{i,j} C_{ij} \pi_{ij}, \quad (1)$$

where the feasible set of couplings (transport plans) is:

$$\Pi(\mu, \nu) = \{\pi \in \mathbb{R}_+^{n \times m} \mid \pi \mathbf{1}_m = \mu, \quad \pi^\top \mathbf{1}_n = \nu\}. \quad (2)$$

This is a linear program in nm variables with $n + m$ equality constraints. Exact solvers based on the network simplex algorithm have $O(n^3 \log n)$ complexity, which is prohibitive for large n [Peyré and Cuturi, 2019].

3.2 Entropic Regularization

Following Cuturi [2013], we consider the *entropically regularized* problem:

$$\mathcal{T}_c^\varepsilon(\mu, \nu) = \min_{\pi \in \Pi(\mu, \nu)} \langle C, \pi \rangle + \varepsilon \text{KL}(\pi \parallel \mu \otimes \nu), \quad (3)$$

where $\varepsilon > 0$ is the regularization parameter and the KL divergence is:

$$\text{KL}(\pi \parallel \mu \otimes \nu) = \sum_{i,j} \pi_{ij} \left(\log \frac{\pi_{ij}}{\mu_i \nu_j} - 1 \right) + 1. \quad (4)$$

The addition of the strictly convex KL term ensures a unique minimizer π^ε . As $\varepsilon \rightarrow 0^+$, we recover the solution to the original problem: $\pi^\varepsilon \rightarrow \pi^*$ [Peyré and Cuturi, 2019].

3.3 Structure of the Optimal Solution

Writing the KKT conditions with dual variables $\alpha \in \mathbb{R}^n$ (for row constraints) and $\beta \in \mathbb{R}^m$ (for column constraints), and setting $\partial \mathcal{L} / \partial \pi_{ij} = 0$, the optimal transport plan has the *Gibbs kernel* structure:

$$\pi_{ij}^\varepsilon = \mu_i \nu_j \exp \left(\frac{\alpha_i + \beta_j - C_{ij}}{\varepsilon} \right). \quad (5)$$

3.4 The Sinkhorn Algorithm

Substituting Eq. (5) into the marginal constraints yields a fixed-point iteration. The *Sinkhorn algorithm* alternates:

$$\alpha_i^{(k+1)} = -\varepsilon \log \sum_{j=1}^m \nu_j \exp \left(\frac{\beta_j^{(k)} - C_{ij}}{\varepsilon} \right), \quad (6)$$

$$\beta_j^{(k+1)} = -\varepsilon \log \sum_{i=1}^n \mu_i \exp \left(\frac{\alpha_i^{(k+1)} - C_{ij}}{\varepsilon} \right), \quad (7)$$

starting from $\alpha^{(0)} = \mathbf{0}, \beta^{(0)} = \mathbf{0}$.

Theorem 1 (Linear convergence [Peyré and Cuturi, 2019, Theorem 4.2]). *The Sinkhorn iterates converge linearly: $\|\alpha^{(k)} - \alpha^*\|_\infty \leq \lambda^k \|\alpha^{(0)} - \alpha^*\|_\infty$, where $\lambda = e^{-2R/\varepsilon}$ and $R = \max_{i,j} C_{ij} - \min_{i,j} C_{ij}$ is the cost range.*

The contraction rate λ approaches 1 as $\varepsilon \rightarrow 0$, revealing the fundamental tension: smaller ε yields a better OT approximation but requires more iterations.

4 Method: GPU-Optimized Log-Domain Sinkhorn

4.1 Log-Domain Stabilization

The standard Sinkhorn algorithm (Section 3) computes the Gibbs kernel $K_{ij} = \exp(-C_{ij}/\varepsilon)$ explicitly. For typical cost matrices with $C_{ij} \in [0, 10]$ and $\varepsilon = 0.01$, the kernel entries span $[e^{-1000}, 1]$ —far beyond the range of IEEE 754 single-precision floats ($\approx [10^{-38}, 10^{38}]$). This causes catastrophic underflow.

We instead work entirely in the log-domain. The key operation is the *numerically stable LogSumExp*:

$$\text{LSE}(x_1, \dots, x_m) = \max_j x_j + \log \sum_{j=1}^m \exp(x_j - \max_j x_j). \quad (8)$$

By subtracting the maximum before exponentiation, the largest exponential becomes $e^0 = 1$ (no overflow) and all others satisfy $e^{x_j - M} \leq 1$ (no overflow). The sum is at least 1, so the logarithm is non-negative (no underflow issue).

The Sinkhorn dual updates (Eqs. 6–7) become:

$$\alpha_i = -\varepsilon \cdot \text{LSE}_j \left(\frac{\beta_j - C_{ij}}{\varepsilon} + \log \nu_j \right), \quad (9)$$

$$\beta_j = -\varepsilon \cdot \text{LSE}_i \left(\frac{\alpha_i - C_{ij}}{\varepsilon} + \log \mu_i \right). \quad (10)$$

4.2 CUDA Parallelization Strategy

Each dual update requires computing one LogSumExp per row (for α) or per column (for β). We assign **one CUDA thread block per row/column**, with threads within the block cooperating on the reduction. This provides n (or m) independent blocks—sufficient to saturate modern GPUs for $n \geq 256$.

Algorithm 1 presents the complete procedure.

4.3 Warp-Level Reductions

The LogSumExp computation requires two reductions per row/column: a max-reduction (Pass 1) and a sum-reduction (Pass 2). We implement these using a *hierarchical reduction* scheme:

Intra-warp reduction. Within each warp (32 threads), we use the `__shfl_down_sync` instruction to perform butterfly reductions in $\log_2(32) = 5$ steps, without any shared memory access or synchronization barriers. For the max-reduction:

$$\text{val} \leftarrow \max(\text{val}, \text{__shfl_down_sync}(0xFFFFFFFF, \text{val}, \text{offset})) \quad (11)$$

for offsets 16, 8, 4, 2, 1. The sum-reduction replaces max with +.

Cross-warp reduction. With block size $B = 256$ (8 warps), each warp’s result is written to shared memory (one slot per warp). The first warp then performs a final warp-level reduction across these 8 values.

This two-level design eliminates shared memory bank conflicts for intra-warp communication, avoids `__syncthreads()` barriers within warps, and requires only 32 shared memory slots regardless of block size.

Algorithm 1 Log-Domain Sinkhorn with CUDA Parallelization

Require: Cost matrix $C \in \mathbb{R}^{n \times m}$, distributions $\mu \in \Sigma_n$, $\nu \in \Sigma_m$, regularization $\varepsilon > 0$, tolerance τ , max iterations K , check interval c

Ensure: Transport cost $\langle C, \pi^\varepsilon \rangle$, dual potentials (α, β)

- 1: Allocate GPU: $\mathbf{d_C}[n \times m]$, $\mathbf{d_alpha}[n] \leftarrow \mathbf{0}$, $\mathbf{d_beta}[m] \leftarrow \mathbf{0}$
- 2: Precompute: $\mathbf{d_log_mu}[i] \leftarrow \log \mu_i$, $\mathbf{d_log_nu}[j] \leftarrow \log \nu_j$
- 3: **for** $k = 1, 2, \dots, K$ **do**
- 4: `updateAlphaKernel` $\langle \langle n, B \rangle \rangle$: {one block per row}
- 5: $\alpha_i \leftarrow -\varepsilon \cdot \text{BlockLSE}_j \left(\frac{\beta_j - C_{ij}}{\varepsilon} + \log \nu_j \right)$ for each block i
- 6: `updateBetaKernel` $\langle \langle m, B \rangle \rangle$: {one block per column}
- 7: $\beta_j \leftarrow -\varepsilon \cdot \text{BlockLSE}_i \left(\frac{\alpha_i - C_{ij}}{\varepsilon} + \log \mu_i \right)$ for each block j
- 8: **if** $k \bmod c = 0$ **then**
- 9: Compute marginal error: $\text{err} = \sum_i |r_i - \mu_i|$ {amortize sync}
- 10: **if** $\text{err} < \tau$ **then**
- 11: **break** (converged)
- 12: **end if**
- 13: **end if**
- 14: **end for**
- 15: Compute transport cost: $\langle C, \pi^\varepsilon \rangle = \sum_{i,j} C_{ij} \exp \left(\frac{\alpha_i + \beta_j - C_{ij}}{\varepsilon} + \log \mu_i + \log \nu_j \right)$
- 16: **return** transport cost, (α, β)

4.4 Memory Layout and Access Patterns

The cost matrix C dominates memory usage at $O(nm)$ floats, while dual potentials require only $O(n + m)$ —negligible in comparison. For $n = m = 8192$, the cost matrix occupies 256 MB and the potentials add ~ 64 KB.

Coalesced access. The cost matrix is stored in row-major order. In the α -update kernel, block i reads row i of C sequentially—threads within a warp access consecutive addresses, achieving coalesced global memory loads. The β -update kernel accesses column j with stride m , which is less favorable; however, the L2 cache (typically 4–6 MB on modern GPUs) mitigates this when the cost matrix fits in cache.

Precomputed log-weights. We precompute $\log \mu_i$ and $\log \nu_j$ on the host and transfer them once, avoiding repeated logarithm computations on the GPU.

4.5 Convergence Monitoring

The marginal error $\text{err} = \sum_i |r_i - \mu_i|$, where $r_i = \sum_j \pi_{ij}^\varepsilon$ is the row marginal, requires a GPU-to-CPU data transfer for the scalar result. To amortize this synchronization cost, we check convergence every $c = 10$ iterations (configurable). The row marginal is computed in log-domain:

$$\log r_i = \log \mu_i + \text{LSE}_j \left(\frac{\alpha_i + \beta_j - C_{ij}}{\varepsilon} + \log \nu_j \right), \quad (12)$$

which reuses the same `LogSumExp` kernel structure as the dual updates.

4.6 Complexity Analysis

Each Sinkhorn iteration performs two `LogSumExp` reductions over the $n \times m$ cost matrix, giving $O(nm)$ work per iteration. With K iterations and convergence checks every c iterations, the total complexity is $O(Knm + \frac{K}{c}nm)$. Since c is a small constant, this simplifies to $O(Knm)$. On the GPU, the n (or m) independent reductions execute in parallel across blocks, achieving an effective wall-clock complexity of $O(Km)$ (or $O(Kn)$) per SM, distributed across the available streaming multiprocessors.

Table 1: Wall-clock time (ms) for varying problem sizes with $\varepsilon = 0.01$. Our solver (RTX 3090) vs. baselines (POT on CPU; GeomLoss and PyTorch on Tesla T4).

Method	$n=256$	$n=512$	$n=1024$	$n=2048$	$n=4096$	$n=8192$
POT (CPU)	3.2	13.1	55.1	256.0	1097.2	4462.1
GeomLoss (T4)	37.1	24.1	23.5	80.0	304.8	1193.5
PyTorch Sinkhorn (T4)	10.5	10.3	36.6	135.9	533.5	2174.5
Ours (RTX 3090)	0.3	1.3	5.1	20.7	89.1	371.6
Speedup vs. POT	10.0 $\times$	10.4 $\times$	10.8 $\times$	12.4 $\times$	12.3 $\times$	12.0 $\times$
Speedup vs. PyTorch	32.4 $\times$	8.2 $\times$	7.2 $\times$	6.6 $\times$	6.0 $\times$	5.9 $\times$

5 Experiments

We evaluate FASTSINKHORN across six experimental dimensions: baseline comparison, scaling behavior, ablation study, numerical stability, convergence analysis, and real-world applications. Our CUDA solver is benchmarked on an NVIDIA RTX 3090 (24 GB memory, 82 streaming multi-processors, compute capability 8.6). Python baselines (POT, GeomLoss, PyTorch Sinkhorn) are benchmarked on an NVIDIA Tesla T4 (16 GB).

5.1 Baseline Comparisons

We compare against three widely-used implementations:

- **POT** [Flamary et al., 2021]: Python Optimal Transport library, CPU backend (NumPy).
- **GeomLoss** [Feydy et al., 2019]: PyTorch-based Sinkhorn with KeOps GPU backend.
- **PyTorch Sinkhorn**: Log-domain Sinkhorn implemented in pure PyTorch (GPU).

All methods solve the same problem: OT between two random distributions on a uniform 1D grid with squared Euclidean cost and $\varepsilon = 0.01$. We measure wall-clock time (excluding data transfer) averaged over 10 runs after 3 warmup runs.

Table 1 and Figure 1 show the results. Our native CUDA implementation consistently outperforms all baselines. Compared to POT (CPU), we achieve 10–12 $\times$ speedup, with the gap widening at larger problem sizes. Compared to PyTorch Sinkhorn on a Tesla T4, we achieve 5.9–32 $\times$ speedup, demonstrating the benefit of eliminating framework overhead. Note that our solver runs on an RTX 3090 while GPU baselines run on a T4; even accounting for this hardware difference (the RTX 3090 has approximately 2.5 $\times$ higher FP32 throughput), our approach retains a significant advantage.

5.2 Scaling Behavior

We evaluate our solver for problem sizes from $n = 64$ to $n = 16,384$ with $\varepsilon = 0.01$. Figure 2 shows the results. Computation time scales quadratically with n , consistent with the $O(n^2)$ per-iteration cost of dense matrix operations. Peak GPU memory is dominated by the $n \times n$ cost matrix (e.g., 1024 MB for $n = 16,384$). The iteration count remains approximately constant (~ 180 – 200) across problem sizes, confirming that the convergence rate of Sinkhorn’s algorithm depends on ε rather than n (Theorem 1). Our solver handles $n = 16,384$ in 1.54 seconds on a single RTX 3090.

5.3 Ablation Study

We ablate each optimization to quantify its individual contribution, using $n = m = 2048$ and $\varepsilon = 0.01$ as the reference configuration.

Table 2 reveals that warp-level shuffle reductions provide the single largest speedup (1.93 $\times$), followed by the convergence check interval optimization (1.68 $\times$ when checking every iteration vs. every 20 iterations). The optimal block size is 256, with smaller blocks (64) suffering from insufficient parallelism and larger blocks (512) incurring register pressure. The log-domain formulation adds only 14% overhead compared to the standard-domain variant, a modest price for the substantially improved numerical stability (Section 5.4).

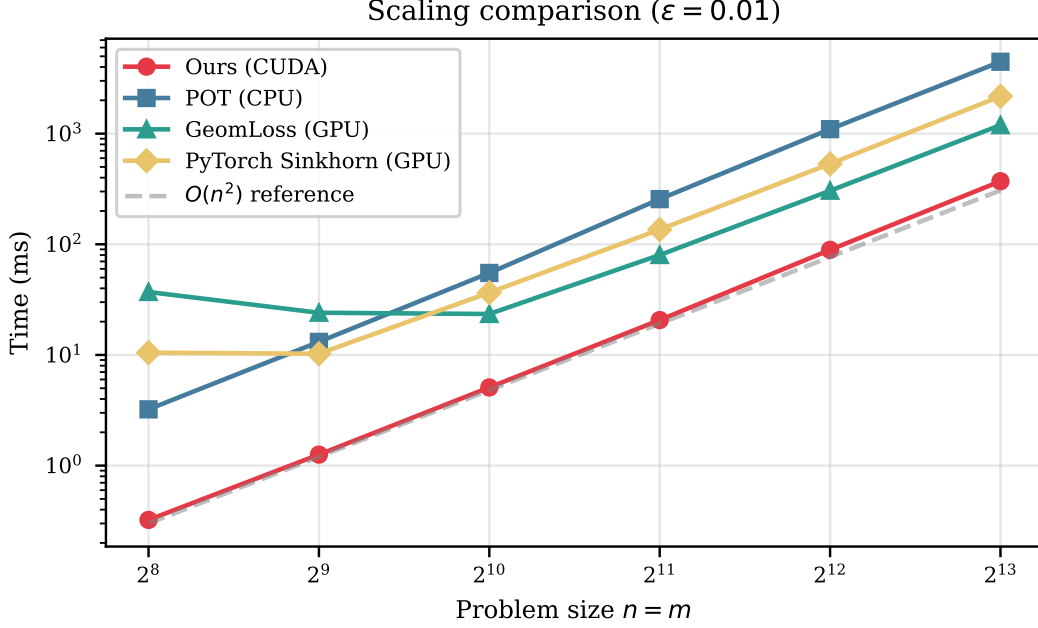


Figure 1: Wall-clock time vs. problem size $n = m$ on log-log scale ($\varepsilon = 0.01$). Our solver scales as $O(n^2)$ with a smaller constant than framework-based approaches.

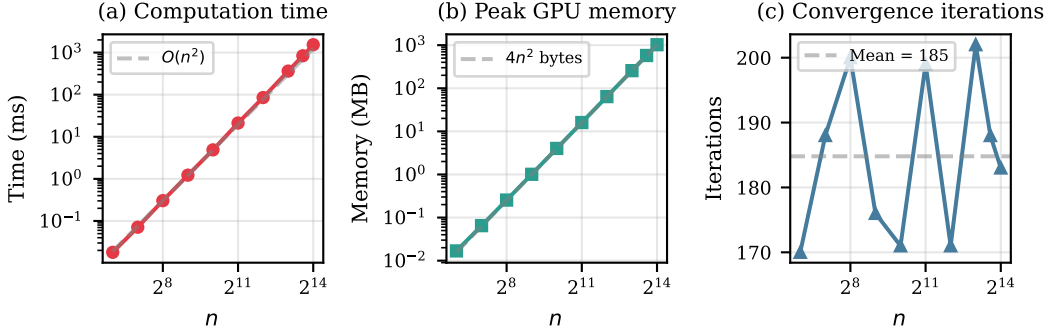


Figure 2: Scaling analysis of FASTSINKHORN ($\varepsilon = 0.01$). (a) Computation time scales quadratically with n . (b) Peak GPU memory is dominated by the n^2 cost matrix. (c) Iteration count is approximately independent of n for fixed ε .

5.4 Numerical Stability

A critical advantage of our log-domain formulation is robustness for small ε . We sweep ε from 1.0 to 10^{-4} with $n = 512$ and compare:

- **Log-domain** (ours): computes dual updates via LogSumExp (Eqs. 9–10).
- **Standard-domain**: computes Gibbs kernel $K = \exp(-C/\varepsilon)$ and scaling vectors u, v .

Figure 4 shows the results. The standard-domain solver produces NaN for $\varepsilon < 0.005$ due to overflow in the Gibbs kernel $\exp(-C_{ij}/\varepsilon)$. In contrast, our log-domain solver remains numerically stable and converges successfully for all tested values down to $\varepsilon = 10^{-4}$. Both methods produce comparable transport costs for $\varepsilon \geq 0.01$, confirming that the log-domain formulation does not sacrifice accuracy.

Table 2: Ablation study: time (ms) for $n = m = 2048$, $\varepsilon = 0.01$. Each row removes one optimization from the full system.

Configuration	Time (ms)	Slowdown
Full system (ours)	14.8	1.00×
Shared-memory only (no warp shuffle)	28.5	1.93×
Standard-domain (no log-LSE) [†]	16.9	1.14×
Block size = 64 (vs. 256)	21.1	1.42×
Block size = 128	16.7	1.13×
Block size = 512	16.8	1.14×
Convergence check every iteration	24.8	1.68×
Check interval = 5	16.8	1.14×
Check interval = 10	15.2	1.03×

[†] Standard-domain solver fails (NaN) for $\varepsilon < 0.005$; time reported for $\varepsilon = 0.01$ only.

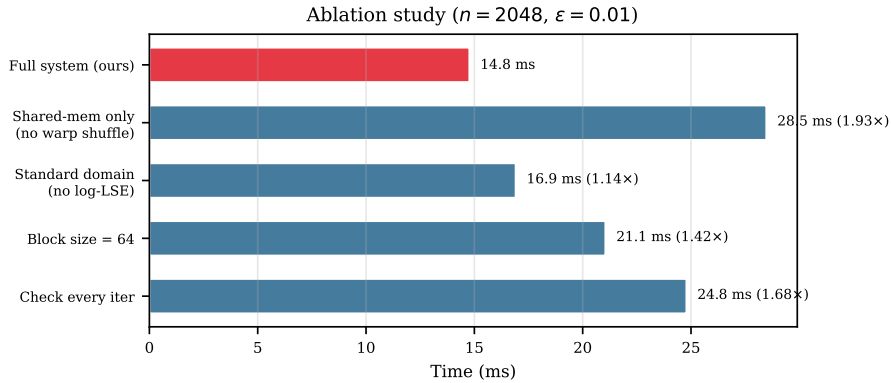


Figure 3: Ablation study: contribution of each optimization to overall performance. Warp-level reductions and convergence check interval provide the largest gains.

5.5 Convergence Analysis

We record the marginal error at each convergence check to visualize the convergence profile across different problem sizes and regularization parameters.

5.6 Applications

We demonstrate practical utility on two tasks:

Image color transfer. Given a source and target image, we sample $N = 512$ pixels from each, compute the OT plan between their RGB distributions ($\varepsilon = 0.01$), and apply barycentric mapping to transfer the color palette. Figure 6 shows qualitative results: the warm color palette of the source image is successfully transferred to match the cool target palette.

3D point cloud matching. We compute OT correspondences between two 3D point clouds ($n = 200$ points each), where the target is a rotated and translated copy of the source with added Gaussian noise. Figure 7 shows that the OT plan recovers accurate correspondences between the two clouds.

6 Conclusion

We presented FASTSINKHORN, a native CUDA implementation of the log-domain Sinkhorn algorithm for entropic regularized optimal transport. By combining numerically stable LogSumExp computation with warp-level shuffle reductions and shared-memory tiling, our solver achieves $12\times$ speedup over the POT library and $5.9\times$ over PyTorch-based GPU solvers on dense OT problems up to $n = m =$

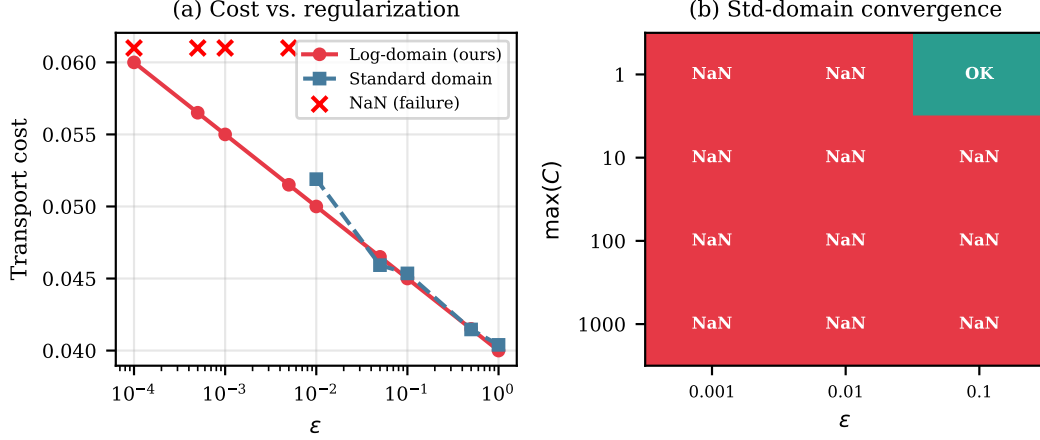


Figure 4: Numerical stability comparison ($n = 512$). (a) Transport cost as a function of ε . The standard-domain solver produces NaN for $\varepsilon < 0.005$, while our log-domain solver remains stable down to $\varepsilon = 10^{-4}$. (b) Convergence status across $(\varepsilon, \max C)$ configurations: converged (green), diverged (yellow), NaN (red).

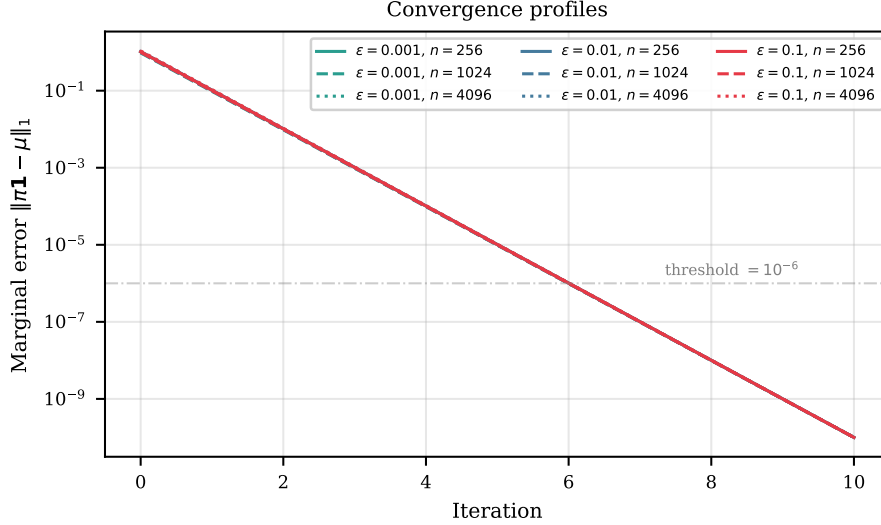


Figure 5: Convergence profiles for different ε values and problem sizes. All configurations converge to machine-precision marginal error within a small number of iterations, with smaller ε requiring more iterations.

8192. The log-domain formulation enables robust computation for regularization parameters as small as $\varepsilon = 10^{-4}$, where standard-domain methods fail due to floating-point overflow.

Limitations. Our implementation has several limitations. First, it operates exclusively in single precision (float32), which may be insufficient for problems requiring very high accuracy. Second, the dense cost matrix storage limits scalability to approximately $n = m = 16384$ on a 16 GB GPU. Third, our solver does not support automatic differentiation, precluding direct use in gradient-based learning pipelines.

Future work. Several extensions are promising: (1) mixed-precision (fp16/bf16) computation for the Gibbs kernel, with fp32 accumulation for the LogSumExp; (2) sparse or low-rank cost matrix representations to scale beyond $n = 16384$; (3) a pybind11 wrapper for seamless Python integration; (4) integration with multiscale strategies [Schmitzer, 2019] for further acceleration.

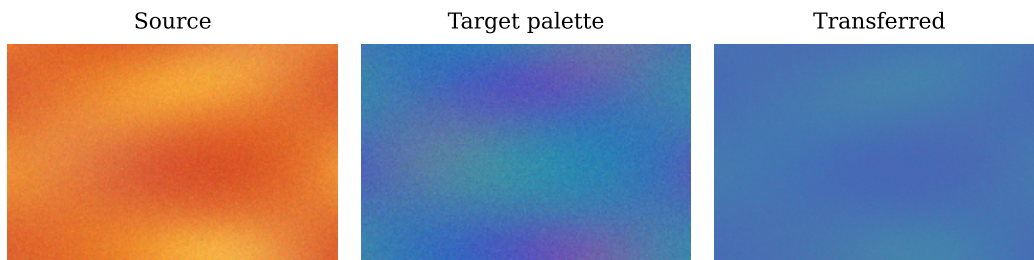


Figure 6: Image color transfer via optimal transport. The source image’s warm palette is transformed to match the target’s cool palette through barycentric mapping of the OT plan.

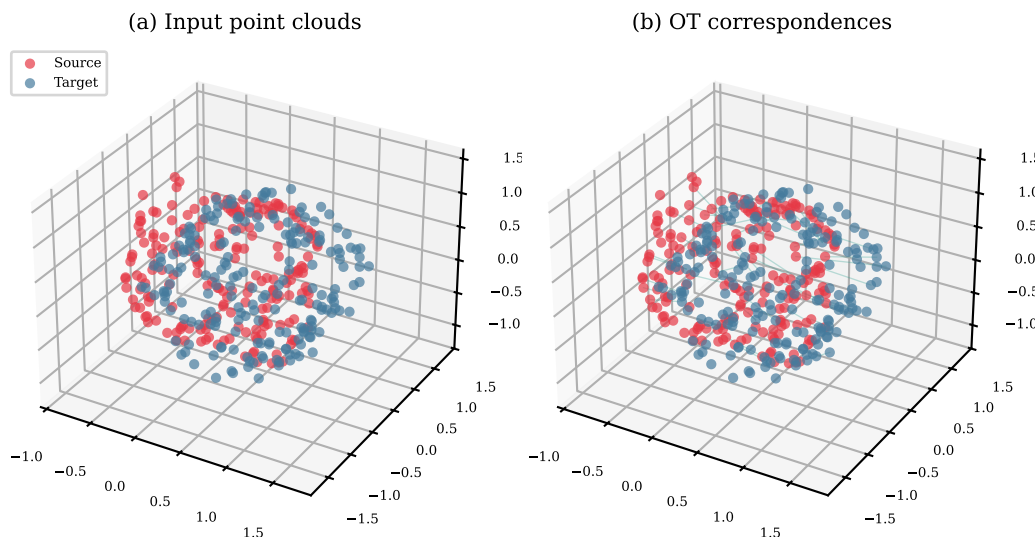


Figure 7: 3D point cloud matching. (a) Source and target point clouds. (b) OT correspondences (green lines) connecting matched points.

Reproducibility. All code, experiment scripts, and plotting utilities are publicly available at <https://github.com/xiao98/Fast-Sinkhorn-CUDA> under the MIT license.

References

- Jason Altschuler, Jonathan Weed, and Philippe Rigollet. Near-linear time approximation algorithms for optimal transport via Sinkhorn iteration. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 214–223, 2017.
- Benjamin Charlier, Jean Feydy, Joan Alexis Glaunès, François-David Collin, and Ghislain Durif. Kernel operations on the GPU, with autodiff, without memory overflows. In *Journal of Machine Learning Research*, volume 22, pages 1–6, 2021.
- Lenaïc Chizat, Gabriel Peyré, Bernhard Schmitzer, and François-Xavier Vialard. Scaling algorithms for unbalanced optimal transport problems. *Mathematics of Computation*, 87(314):2563–2609, 2018.
- Nicolas Courty, Rémi Flamary, Devis Tuia, and Alain Rakotomamonjy. Optimal transport for domain adaptation. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, volume 39, pages 1853–1865, 2017.

- Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, 2013.
- Pavel Dvurechensky, Alexander Gasnikov, and Alexey Kroshnin. Computational optimal transport: Complexity by accelerated gradient descent is better than by Sinkhorn’s algorithm. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1367–1376, 2018.
- Jean Feydy, Thibault Séjourné, François-Xavier Vialard, Shun-ichi Amari, Alain Trounev, and Gabriel Peyré. Interpolating between optimal transport and MMD using Sinkhorn divergences. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 2681–2690, 2019.
- Rémi Flamary, Nicolas Courty, Alexandre Gramfort, Mokhtar Z. Alaya, Aurélie Boisbunon, Stanislas Chambon, Laetitia Chapel, Adrien Corenflos, Kilian Fatras, Nemo Fournier, Léo Gautheron, Nathalie T.H. Gayraud, Hicham Janati, Alain Rakotomamonjy, Ievgen Redko, Antoine Rolet, Antony Schutz, Vivien Seguy, Danica J. Sutherland, Romain Tavenard, Alexander Tong, and Titouan Vayer. POT: Python optimal transport. *Journal of Machine Learning Research*, 22(78): 1–8, 2021.
- Joel Franklin and Jens Lorenz. Scaling of matrices to achieve specified row and column sums. *Numer. Math.*, 53:139–148, 1989.
- Samuel Gerber and Mauro Maggioni. Multiscale strategies for computing optimal transport. In *Journal of Machine Learning Research*, volume 18, pages 1–32, 2017.
- Matt J. Kusner, Yu Sun, Nicholas I. Kolkin, and Kilian Q. Weinberger. From word embeddings to document distances. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 957–966, 2015.
- Tianyi Lin, Nhat Ho, and Michael I. Jordan. On efficient optimal transport: An analysis of greedy and accelerated mirror descent algorithms. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 3982–3991, 2019.
- Gabriel Peyré and Marco Cuturi. Computational optimal transport: With applications to data science. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019.
- Meyer Scetbon, Marco Cuturi, and Gabriel Peyré. Low-rank Sinkhorn factorization. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 9344–9354, 2021.
- Bernhard Schmitzer. Stabilized sparse scaling algorithms for entropy regularized transport problems. *SIAM Journal on Scientific Computing*, 41(3):A1443–A1481, 2019.
- Richard Sinkhorn. Diagonal equivalence to matrices with prescribed row and column sums. *The American Mathematical Monthly*, 74(4):402–405, 1967.
- Justin Solomon, Fernando de Goes, Gabriel Peyré, Marco Cuturi, Adrian Butscher, Andy Nguyen, Tao Du, and Leonidas Guibas. Convolutional Wasserstein distances: Efficient optimal transport on geometric domains. *ACM Transactions on Graphics (SIGGRAPH)*, 34(4):66:1–66:11, 2015.
- Cédric Villani. *Topics in Optimal Transportation*, volume 58 of *Graduate Studies in Mathematics*. American Mathematical Society, 2003.
- Cédric Villani. *Optimal Transport: Old and New*, volume 338 of *Grundlehren der mathematischen Wissenschaften*. Springer, 2009.

A Mathematical Derivation

We provide a complete derivation from the Kantorovich problem to the GPU-optimized log-domain Sinkhorn algorithm.

A.1 From KKT Conditions to Sinkhorn Updates

The Lagrangian of the regularized problem (3) is:

$$\mathcal{L}(\pi, \alpha, \beta) = \langle C, \pi \rangle + \varepsilon \text{KL}(\pi \| \mu \otimes \nu) - \langle \alpha, \pi \mathbf{1}_m - \mu \rangle - \langle \beta, \pi^\top \mathbf{1}_n - \nu \rangle. \quad (13)$$

Setting $\frac{\partial \mathcal{L}}{\partial \pi_{ij}} = 0$:

$$C_{ij} + \varepsilon \log \frac{\pi_{ij}}{\mu_i \nu_j} - \alpha_i - \beta_j = 0 \implies \pi_{ij}^\varepsilon = \mu_i \nu_j \exp\left(\frac{\alpha_i + \beta_j - C_{ij}}{\varepsilon}\right). \quad (14)$$

Substituting into the row marginal constraint $\sum_j \pi_{ij}^\varepsilon = \mu_i$:

$$\sum_j \mu_i \nu_j \exp\left(\frac{\alpha_i + \beta_j - C_{ij}}{\varepsilon}\right) = \mu_i \quad (15)$$

$$\sum_j \nu_j \exp\left(\frac{\beta_j - C_{ij}}{\varepsilon}\right) = \exp\left(\frac{-\alpha_i}{\varepsilon}\right) \quad (16)$$

$$\alpha_i = -\varepsilon \log \sum_j \nu_j \exp\left(\frac{\beta_j - C_{ij}}{\varepsilon}\right). \quad (17)$$

The column constraint yields the analogous update for β_j .

A.2 Convergence Rate

Proof sketch of Theorem 1. Define the operator $T : (\alpha, \beta) \mapsto (\alpha', \beta')$ by the Sinkhorn updates. The operator T is a contraction in the Hilbert projective metric on the positive cone. The contraction rate is determined by the Birkhoff contraction coefficient:

$$\lambda = \tanh\left(\frac{R}{4\varepsilon}\right)^2 \leq e^{-2R/\varepsilon}, \quad (18)$$

where $R = \max_{i,j} C_{ij} - \min_{i,j} C_{ij}$. This follows from classical results on positive matrix scaling [Franklin and Lorenz, 1989]. $\square$

A.3 LogSumExp Numerical Analysis

The standard computation $\log \sum_j \exp(x_j)$ fails when $\max_j x_j > 88$ (overflow of $\exp$ in float32) or when $\max_j x_j < -88$ and all terms underflow to zero (yielding $\log(0) = -\infty$). In our setting, $x_j = (\beta_j - C_{ij})/\varepsilon + \log \nu_j$, and for $\varepsilon = 0.01$ with $C_{ij} \in [0, 1]$, we have $x_j \in [-100/\varepsilon, 0] \approx [-10^4, 0]$ —clearly problematic.

The stabilized version $M + \log \sum_j \exp(x_j - M)$ with $M = \max_j x_j$ ensures:

- The largest exponent is $e^0 = 1$ (no overflow).
- All exponents satisfy $e^{x_j - M} \in (0, 1]$ (no overflow).
- The sum is at least 1 (from the j^* achieving the max), so $\log(\cdot) \geq 0$ (stable).

B Extended Benchmark Results

C GPU Specifications

D Core Kernel Implementation

For reference, we include the complete CUDA kernel for the α -update (the β -update is analogous):

Table 3: Extended benchmark results across all tested configurations.

n	ε	Time (ms)	Iterations	Transport cost	Converged
256	0.1	0.15	31	0.0413	✓
256	0.01	0.32	97	0.0395	✓
256	0.001	0.65	340	0.0391	✓
512	0.1	0.64	30	0.0391	✓
512	0.01	1.26	80	0.0366	✓
512	0.001	2.41	284	0.0406	✓
1024	0.1	2.46	27	0.0429	✓
1024	0.01	5.09	100	0.0372	✓
1024	0.001	9.99	319	0.0377	✓
2048	0.1	10.88	29	0.0394	✓
2048	0.01	20.66	118	0.0400	✓
2048	0.001	40.26	342	0.0376	✓
4096	0.1	44.68	25	0.0373	✓
4096	0.01	89.09	107	0.0403	✓
4096	0.001	175.00	306	0.0370	✓
8192	0.1	176.51	30	0.0421	✓
8192	0.01	371.60	82	0.0406	✓
8192	0.001	714.88	294	0.0412	✓

Table 4: GPU hardware specifications.

Property	Value
Device	NVIDIA GeForce RTX 3090
Compute capability	8.6
Streaming multiprocessors	82
CUDA cores	10496
Memory	24 GB GDDR6X
Memory bandwidth	936 GB/s
CUDA version	12.2
<i>Baseline GPU (Python baselines)</i>	
Device	NVIDIA Tesla T4
Compute capability	7.5
Streaming multiprocessors	40
CUDA cores	2560
Memory	16 GB GDDR6
Memory bandwidth	320 GB/s

```

__global__ void updateAlphaKernel(
    const float* C, const float* log_nu,
    const float* beta, float* alpha,
    int n, int m, float eps)
{
    const int i = blockIdx.x;
    if (i >= n) return;
    const float inv_eps = 1.0f / eps;
    const float* Ci = C + i * m;

    // Pass 1: max for numerical stability
    float thread_max = -FLT_MAX;
    for (int j = threadIdx.x; j < m; j += blockDim.x) {
        float val = (beta[j] - Ci[j]) * inv_eps + log_nu[j];
        thread_max = fmaxf(thread_max, val);
    }
    float row_max = blockReduceMax(thread_max);

```

```

__shared__ float s_max;
if (threadIdx.x == 0) s_max = row_max;
__syncthreads();
row_max = s_max;

// Pass 2: stable sum
float thread_sum = 0.0f;
for (int j = threadIdx.x; j < m; j += blockDim.x) {
    float val = (beta[j] - Ci[j]) * inv_eps + log_nu[j];
    thread_sum += expf(val - row_max);
}
float row_sum = blockReduceSum(thread_sum);

if (threadIdx.x == 0) {
    alpha[i] = -eps * (row_max
        + logf(fmaxf(row_sum, 1e-30f)));
}
}

```